

SISTEMA PÚBLICO DE AGENDAMENTO HOSPITALAR

AgendaSUS

Documentação Técnica — Arquitetura de Software

Daniel Dantas de Farias

Ericlys Severino da Silva

Márcio José da Silva Moraes Junior

Othon Henrique Queiroz de Oliveira

Rodrigo Nunes Peixoto Ramos

01

Filas & Espera

Pacientes enfrentam longas filas presenciais para agendamentos na rede pública de Recife.

02

Informação Descentralizada

Ausência de canal unificado para consultar especialidades, horários e unidades disponíveis.

03

Absenteísmo & Ineficiência

Sem lembretes ou cancelamentos online, vagas são perdidas e recursos médicos desperdiçados.

→ A ausência de uma interface centralizada compromete o planejamento estratégico da saúde municipal.

Mediador digital entre o cidadão recifense e a infraestrutura de saúde pública municipal.



Filtro Avançado

Busca por especialidade, proximidade geográfica e disponibilidade de horários.



Agendamento Dinâmico

Marcação remota de consultas, sem necessidade de deslocamento prévio.



Painel de Verificação

Acompanhe status, receba lembretes e cancele consultas online.



Mapeamento de Unidades

Clínicas com especialidades, horários e contatos.

Arquitetura — Padrão MVC

VIEW

React · SPA

Interface responsiva, filtros e painéis do usuário



Requisições HTTP / REST

CONTROLLER

Python + Flask · API RESTful

Rotas, autenticação, validação de regras de negócio



ORM / Consultas SQL

MODEL

MySQL · ORM / SQL

Persistência de dados: pacientes, agendamentos e clínicas



React

Apresentação

Interface responsiva, componentes reutilizáveis e experiência do cidadão.



Python + Flask

Aplicação

Micro-framework leve, RESTful, com Blueprints para modularização.



MySQL

Persistência

SGBD relacional, garantindo integridade dos dados de pacientes e agendamentos.



Docker

Deploy

Containerização de 1 container monolítico enxuto, facilitando a execução do código.

Componentes & Blueprints Flask

1. Divide a aplicação em módulos independentes por domínio: usuários, agendamentos, clínicas.
2. Promove desacoplamento e separação clara de responsabilidades no padrão MVC.
3. Permite que múltiplos desenvolvedores trabalhem em paralelo sem conflitos.
4. Funciona como componente plugável: registrado na aplicação principal com prefixo de URL próprio.
5. Facilita a manutenção e escalabilidade do código.
5. Ferramenta essencial para organizar aplicações web.

```
from flask import Flask, render_template
from flask_cors import CORS

from controllers.user_controller import user_bp
from controllers.appointment_controller import appointment_bp
from controllers.schedule_controller import schedule_bp
from controllers.clinic_controller import clinic_bp
from controllers.specialtie_controller import specialtie_bp

app = Flask(
    __name__,
    static_folder='../frontend/dist/assets',
    template_folder='../frontend/dist'
)

CORS(app)

@app.route('/')
def index():
    return render_template('index.html')

app.register_blueprint(user_bp, url_prefix='/api')
app.register_blueprint(appointment_bp, url_prefix='/api')
app.register_blueprint(schedule_bp, url_prefix='/api')
app.register_blueprint(clinic_bp, url_prefix='/api')
app.register_blueprint(specialtie_bp, url_prefix='/api')
```

Testes Automatizados — Pytest + Mock

Isolamento

Mocks substituem o banco de dados real nos testes

Velocidade

Valida todas as rotas em milissegundos

Contratos de API

Garante status HTTP e JSON corretos ao frontend

Teste	Arquivo	Objetivo
test_index_route	test_app.py	Valida inicialização do Flask e roteamento da página inicial
test_get_users_api	test_users.py	Certifica que /api/users retorna Status 200 corretamente
test_create_appointment_success	test_appointments.py	Valida criação de agendamento via POST com Mock do banco
test_get_specialties_data_success	test_specialties.py	Confirma payload JSON com especialidades em /api/specialties
test_get_free_schedules_success	test_schedules.py	Testa exposição de horários disponíveis em /api/free_schedules
test_get_filtered / patient_schedules	test_schedules.py	Valida filtros dinâmicos e histórico por paciente via parâmetros URL

```
pytest
```

```
===== test session starts =====  
platform win32 -- Python 3.11.3, pytest-9.0.3, pluggy-1.6.0  
rootdir: C:\Daniel\Uninassau\3o Período\Arquitetura de Software (Clóvis)\Testes\agendasus  
  
collected 8 items  
  
tests\e2e\test_app.py . [ 12%]  
tests\e2e\test_appointments.py .. [ 25%]  
tests\e2e\test_clinics.py . [ 37%]  
tests\e2e\test_schedules.py ... [ 75%]  
tests\e2e\test_specialtie.py . [ 87%]  
tests\e2e\test_users.py . [100%]  
  
===== 8 passed in 0.17s =====
```




Single Container (All-in-One)

Build de Produção do Frontend (React)

Backend (Python + Flask) & Banco (MySQL)

Serviços Centralizados (Comunicação Localhost)

- ✓ Ambientes global unificado e idêntico em todas as máquinas
- ✓ Deploy extremamente simplificado (única imagem)
- ✓ Centralização de infraestrutura com menor overhead



Acessibilidade

Interface inclusiva para cidadãos de todas as faixas etárias e níveis de familiaridade digital.



Confiabilidade

Arquitetura MVC + testes automatizados garantem integridade e contratos de API robustos.



Escalabilidade

Estrutura modular com Docker e Blueprints permite expansão futura sem comprometer o núcleo.

"O AgendaSUS não é apenas uma ferramenta técnica — é uma ponte entre o cidadão recifense e o direito à saúde com dignidade."

Obrigado!

AgendaSUS — Sistema Público de Agendamento Hospitalar

UNINASSAU Olinda · Disciplina: Arquitetura de Software · 2026

Disponíveis para perguntas.